Mini LMS

Plateforme LMS avec Génération IA et Révision Espacée

Stagiaire	Latreche
Période	Avril 2026
Stack	Laravel 11 · MySQL · Tailwind · Alpine.js · Gemini AI
Production	https://latreche-lms.alwaysdata.net

Résumé

Plateforme LMS Laravel 11 avec génération de contenu pédagogique structuré par IA (Gemini), système de flashcards avec algorithme de révision espacée SM-2, enrichissement automatique multi-sources (images Wikimedia/Pexels, sources validées, diagrammes Mermaid) et saisie vocale. Architecture en couches avec autorisation par Policies, isolation stricte des rôles, et gestion d'erreurs indépendante du layout dashboard. Déployée sur AlwaysData.

Contents

1 Contexte et Objectifs **1**
- 1.1 Contexte . . . 1
- 1.2 Objectifs . . . 1

2 Stack Technique **1**

3 Architecture **1**
- 3.1 Flux de requête . . . 1
- 3.2 Inventaire des composants . . . 3

4 Modèle de Données **3**
- 4.1 Colonnes clés . . . 4

5 Authentification & Redirection par Rôle **4**
- 5.1 Insights clés . . . 5

6 Rôles & Permissions **6**

7 Flashcards (Révision Espacée SM-2) **7**
- 7.1 Insights clés . . . 8

8 Module IA - Génération de Contenu **8**
- 8.1 Pipeline . . . 9
- 8.2 Paramètres . . . 10
- 8.3 Type `full` (cours + quiz + flashcards) . . . 10
- 8.4 Contrôle du nombre de chapitres . . . 10
- 8.5 Workflow par rôle . . . 10

9 Pipeline d'Enrichissement **11**

10 Sécurité **12**

11 Déploiement **12**

12 Limitations Connues **13**

13 Améliorations Futures **13**

Conclusion **13**

1. Contexte et Objectifs

1.1 Contexte

Concevoir une plateforme LMS exploitant l'IA générative pour la création automatique de contenu pédagogique structuré (formations, chapitres, quiz, flashcards), avec une isolation stricte entre rôles et un système de révision espacée.

1.2 Objectifs

#	Objectif	Statut
1	Architecture Laravel en couches (Service Layer + Policies + FormRequests)	✓
2	Authentification multi-rôles avec redirection basée sur le rôle et isolation stricte des layouts	✓
3	Quiz step-by-step avec scoring sécurisé (anti-triche, verrou pessimiste)	✓
4	Flashcards avec algorithme SM-2 et distribution automatique sur inscription	✓
5	Génération IA contrôlée : nombre de chapitres, profondeur, type, fichiers joints	✓
6	Pipeline d'enrichissement multi-sources avec fallbacks (images, sources, diagrammes)	✓
7	Parité d'édition admin/étudiant pour les formations générées par l'étudiant	✓
8	Déploiement production sur hébergement mutualisé	✓

2. Stack Technique

Couche	Technologie	Justification
Backend	Laravel 11, PHP 8.2	Eloquent, migrations, policies, FormRequests
BDD	MySQL 8 / SQLite	MySQL prod, SQLite dev local
Frontend	Blade + Tailwind CDN	Server-rendered, pas de build step
Interactivité	Alpine.js	Quiz step-by-step, modales, formulaires réactifs
Diagrammes	Mermaid.js (CDN)	Rendu client-side du code IA, `securityLevel: strict`
Voix	Web Speech API	Saisie native FR/EN/AR, sans backend
IA	Google Gemini API	JSON structuré, multi-modèle fallback, Files API
Images	Wikimedia + Pexels	CC en priorité, fallback paysage
Hébergement	AlwaysData	PHP/MySQL mutualisé

3. Architecture

3.1 Flux de requête

`Request → Auth+Role MW → FormRequest → Controller → Service → Model → DB`

Les contrôleurs sont *thin* : orchestration uniquement (validation déléguée aux `FormRequest`, autorisation aux `Policies`, logique métier aux `Services`). Tout 403/404/500 est rendu via `layouts.error` (layout dédié, sans sidebar) pour qu'aucune erreur ne fuite la chrome du rôle opposé.

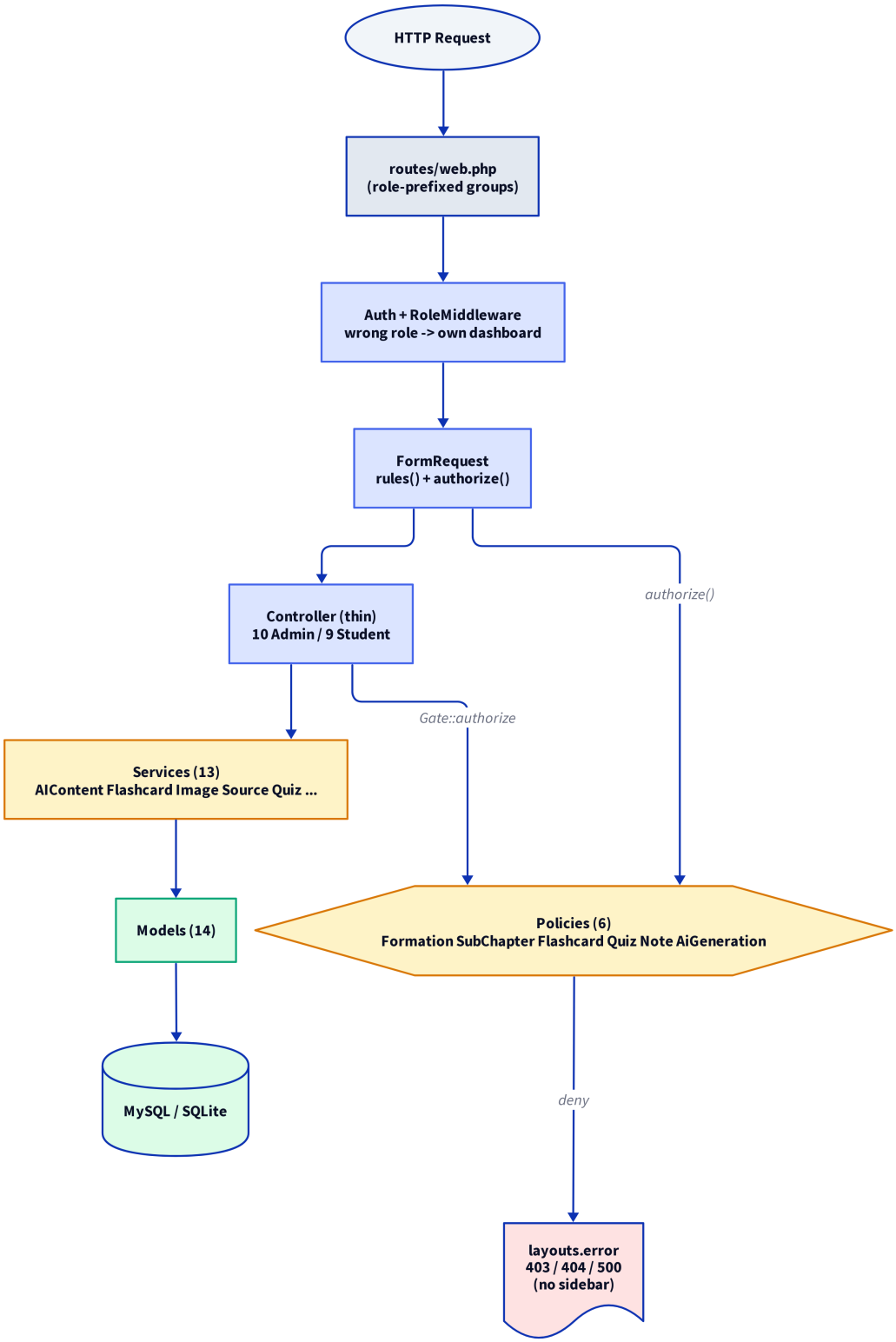

Figure 1: Architecture en couches : isolation des responsabilités, policies en garde sur chaque action sensible

3.2 Inventaire des composants

Composant	N	Rôle
Controllers Admin	10	CRUD complet : formations, chapitres, sous-chapitres, quizzes, questions, users, notes, AI, flashcards
Controllers Student	9	Lecture + CRUD sur formations *possédées*, AI, flashcards, todos
`AuthController`	1	Login/register/logout, redirection par rôle
FormRequests	9	Validation + `authorize()` basée sur les Policies
Policies	6	`Formation`, `SubChapter`, `Flashcard`, `Quiz`, `Note`, `AiGeneration`
Models Eloquent	14	Avec relations, scopes, casts, soft deletes
Services	13	Logique métier, transactions, APIs externes (voir §9)
Migrations	16	Schéma + colonnes enrichissement + flashcards + ownership + CHECK `type='full'`
Vues Blade	60+	Layouts (`app`, `error`), partials, vues admin/student

4. Modèle de Données

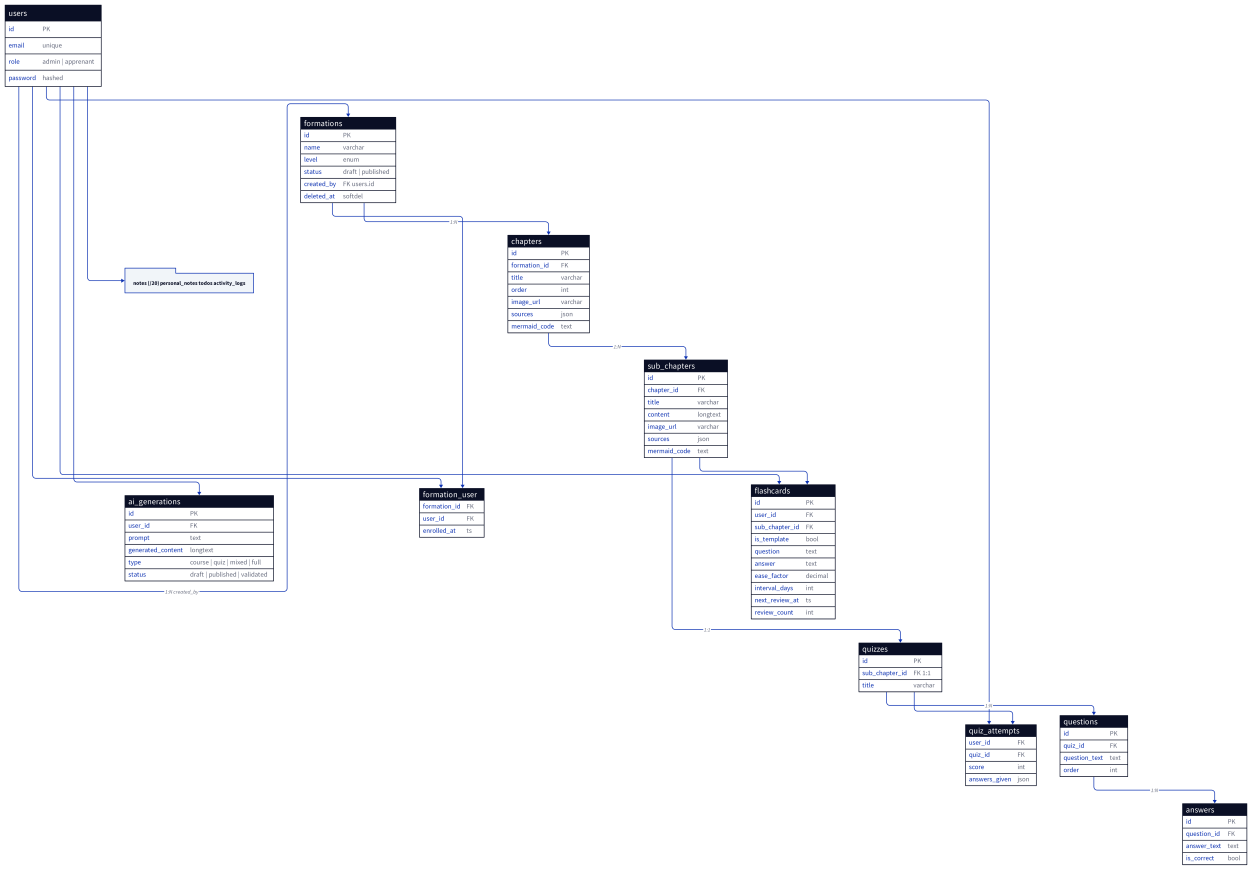

Figure 2: ERD : 14 entités principales (+ tables auxiliaires regroupées), enrichissement IA sur `chapters` et `sub_chapters`, soft delete sur formations, ownership via `created_by`

4.1 Colonnes clés

Colonne	Tables	Usage
`image_url`, `image_credit`	chapters, sub_chapters	Image illustrative + attribution licence
`sources` (JSON)	chapters, sub_chapters	Tableau {title, url, type} validé
`mermaid_code` (TEXT)	chapters, sub_chapters	Diagramme généré par IA
`created_by`	formations	Trace l'auteur (admin ou étudiant) pour ownership
`type` CHECK	ai_generations	`course` | `quiz` | `mixed` | `full` (cours+quiz+flashcards)
`is_template`	flashcards	Sépare cartes de distribution (admin) et cartes étudiables (SM-2)
`ease_factor`, `interval_days`, `next_review_at`	flashcards	État SM-2 par carte étudiable
`validated_at`	ai_generations	Marqueur de validation par l'étudiant

5. Authentification & Redirection par Rôle

Le système d'authentification a été conçu pour éviter le bug classique de redirection inter-rôles : un admin dont la session expire sur `/admin/...`, suivi d'une connexion étudiant, ne doit *jamais* se voir redirigé vers une URL admin et obtenir un 403 *dans le layout admin*.

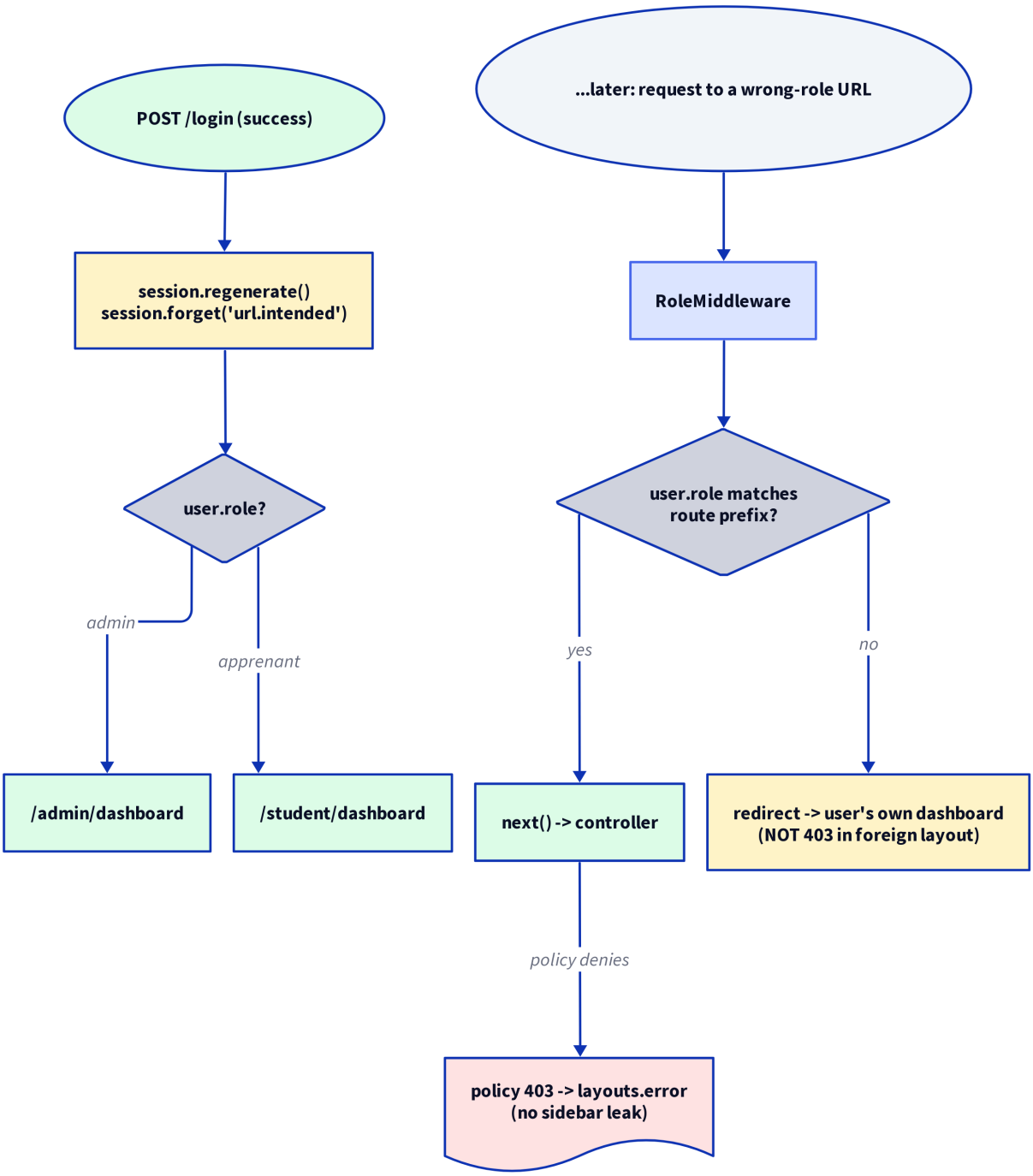

Figure 3: Flux de login + RoleMiddleware : la redirection est pilotée strictement par `user.role`, jamais par `intended()`

5.1 Insights clés

- **Pas d'`intended()` URL** : `AuthController::login` appelle `forget('url.intended')` avant de rediriger, supprimant la cause racine du bug inter-rôles.
- `RoleMiddleware` **redirige plutôt que d'abort 403** : un étudiant qui touche une URL admin est redirigé vers *son* dashboard, pas traité par un 403 dans le layout admin.
- **Layout d'erreur dédié** : si un 403 doit malgré tout apparaître (après un `Gate::authorize`), il s'affiche dans `layouts.error` (sans sidebar, sans chrome de rôle).
- **Session régénérée** à chaque login pour prévenir la fixation.

6. Rôles & Permissions

Action	Admin	Apprenant
Tableau de bord (métriques)	✓	✓
CRUD formations / chapitres / sous-chapitres officiels	✓	×
CRUD complet sur formations *générées par soi-même*	—	✓
Inscription des apprenants à une formation	✓	×
Consultation formations enrollées / publiées	✓	✓
Quiz step-by-step + scoring	×	✓
Génération de quiz sur sous-chapitre existant	✓	×
Notes /20 (administration)	✓	✓ (lecture)
Notes personnelles par sous-chapitre	×	✓
Todos personnelles	×	✓
Flashcards : templates (distribués à l'inscription)	✓	×
Flashcards : copies personnelles étudiables (SM-2)	✓	✓
Génération IA → import en formation officielle	✓	×
Génération IA → validation en formation personnelle	×	✓

Ownership : `formations.created_by` trace l'auteur. La parité d'édition de l'étudiant repose sur `FormationPolicy::updat` et `SubChapterPolicy::update`, qui retournent `true` ssi le user est l'auteur. Les boutons d'édition apparaîssent via `auth()->user()->can('update', $model)`, jamais par vérification de rôle.

Sécurité quiz : la génération de quiz sur un sous-chapitre est réservée à l'admin. Un `Quiz` est lié 1:1 au `SubChapter`, donc un quiz créé par un étudiant sur un sous-chapitre admin serait visible par tous les autres étudiants inscrits — comportement intentionnellement bloqué.

7. Flashcards (Révision Espacée SM-2)

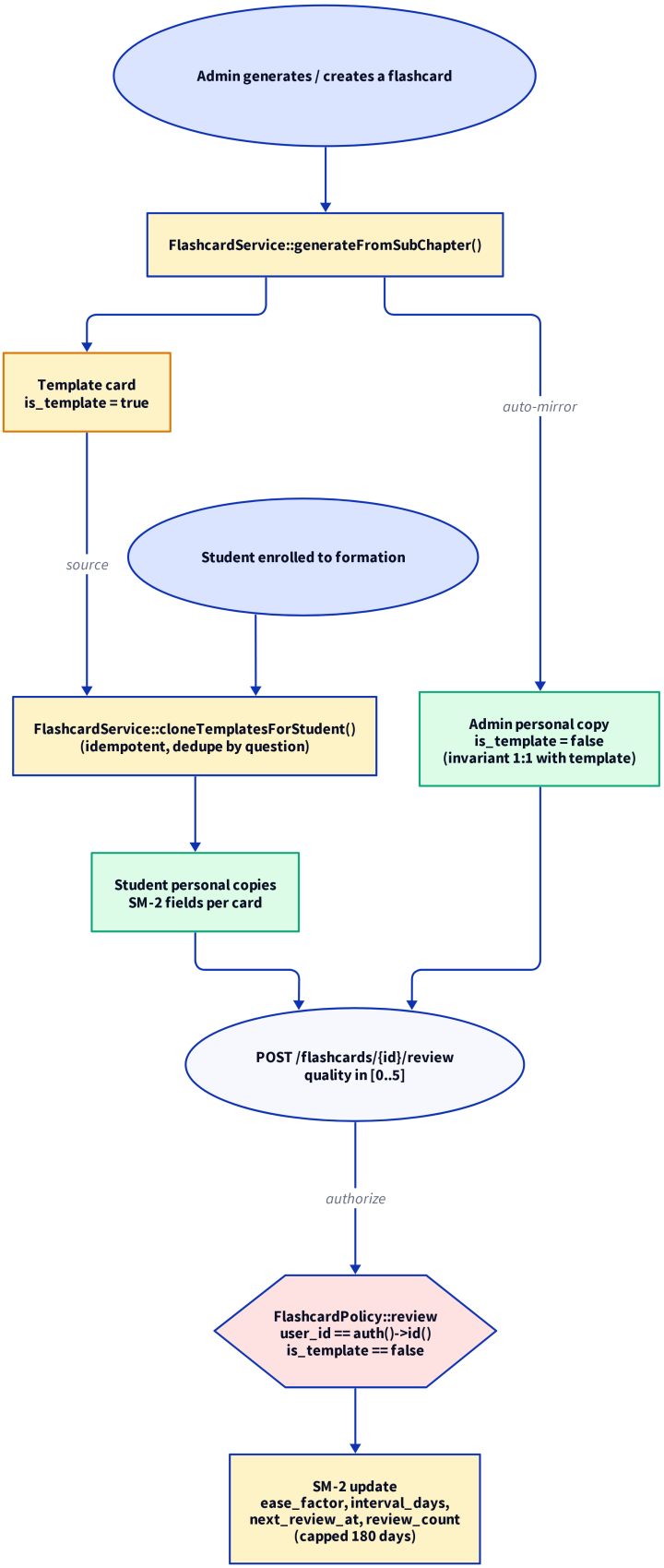

Figure 4: Modèle dual templates / personnelles, clonage automatique à l'inscription, mise à jour SM-2 à chaque révision

7.1 Insights clés

- **Deux types de cartes** via `is_template` : les *templates* (admin) ne sont pas étudiables ; les copies *personnelles* portent l'état SM-2 complet.
- **Invariant admin** : pour chaque template détenu par un admin, il existe une copie personnelle par le même admin (garantie par `FlashcardService` à chaque entrée : `generate`, `store`, `update`, `destroy`). Conséquence : le mode *Test* de l'admin fonctionne toujours.
- **Distribution à l'inscription** : `cloneTemplatesForStudent()` duplique les templates en copies personnelles, idempotent (dédupe par question).
- **SM-2 standard** : `quality` 0-5, mise à jour de `ease_factor`, `interval_days`, `next_review_at`, `review_count`. Intervalle plafonné à 180 jours.
- **Politique stricte** : `FlashcardPolicy::review` exige `user_id === auth()->id()` et `is_template === false` ; aucun risque d'IDOR cross-user ou d'étude des templates.

8. Module IA - Génération de Contenu

8.1 Pipeline

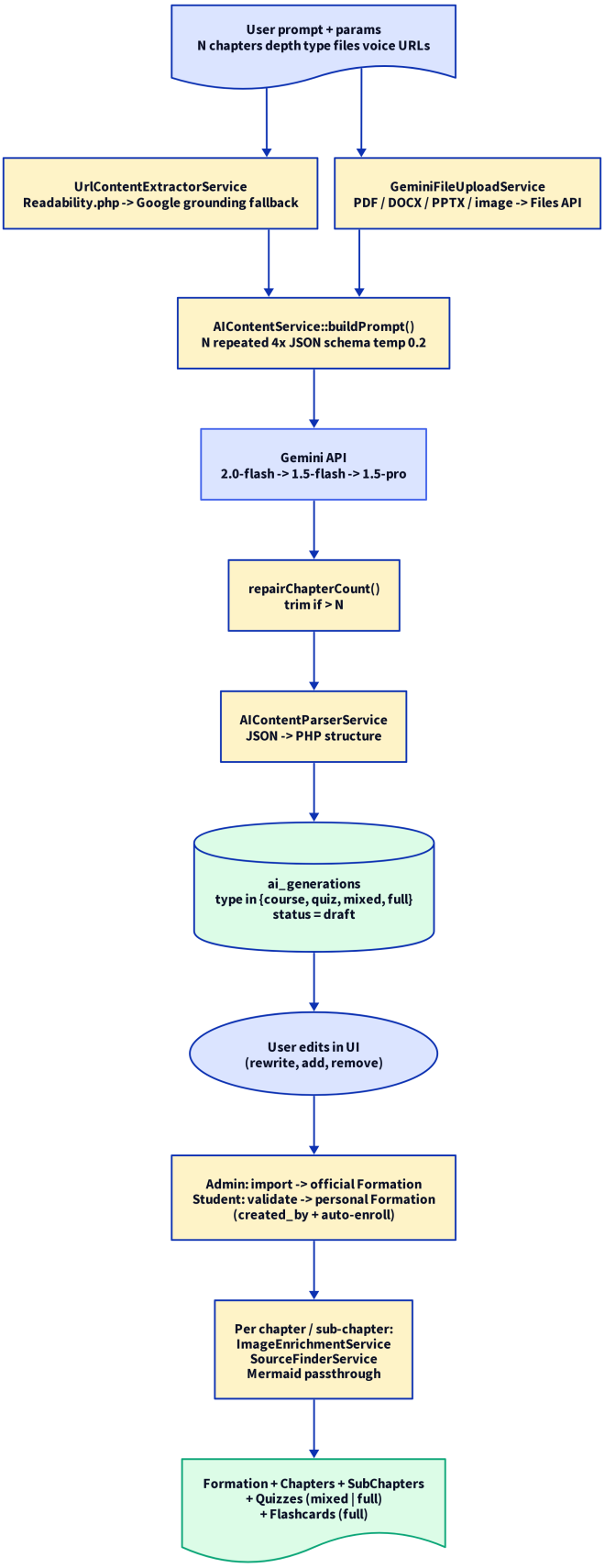

Figure 5: Pipeline complet : prompt → extraction URL/fichiers → Gemini → réparation → édition → import enrichi

8.2 Paramètres

Paramètre	Détail
Modèles fallback	`gemini-2.0-flash` → `1.5-flash` → `1.5-pro`
Température	0.2 (instruction-following), 0.3 (flashcards), 0.4 (rewrite)
Nombre de chapitres	1-10 ; répété 4· dans le prompt + `repairChapterCount()` post-trim
Profondeur	Standard (~300 mots), Détaillé (~600), Exhaustif (1000+) par sous-chapitre
Type	`course`, `quiz`, `mixed`, `full` (cours + quiz + flashcards à l'import)
Pièces jointes	5 max, 25 Mo total (PDF, DOCX, PPTX, images), via Gemini Files API
URLs	Détection auto, extraction Readability.php, fallback Google Search Grounding
Saisie vocale	Web Speech API (FR/EN/AR), append au textarea
Output	JSON forcé (`responseMimeType`), 40000 tokens max

8.3 Type `full` (cours + quiz + flashcards)

Le type `full` est un *marqueur* sur `ai_generations.type` : la génération produit du contenu `mixed` (cours + quiz), puis l'enregistrement est promu à `full` pour signaler à l'importeur de générer aussi des flashcards pour chaque sous-chapitre. La contrainte CHECK SQLite (`type IN {course, quiz, mixed, full}`) a été étendue par migration dédiée pour autoriser cette promotion.

8.4 Contrôle du nombre de chapitres

#	Mécanisme	Rôle
1	Prompt unifié	Suppression du split system/user
2	Répétition 4·	N rappelé dans règles, schema, exemple, footer
3	Température 0.2	Conformité > créativité
4	JSON mode forcé	`responseMimeType: application/json`
5	Réparation post-gen	`count(chapters) > N` → trim à N

8.5 Workflow par rôle

Rôle	Transitions
Admin	`draft` → édition → `import` → `published` (Formation officielle, `created_by` = admin)
Apprenant	`draft` → édition → `validate` → `validated` (Formation personnelle, `created_by` = étudiant, auto-inscription)

9. Pipeline d'Enrichissement

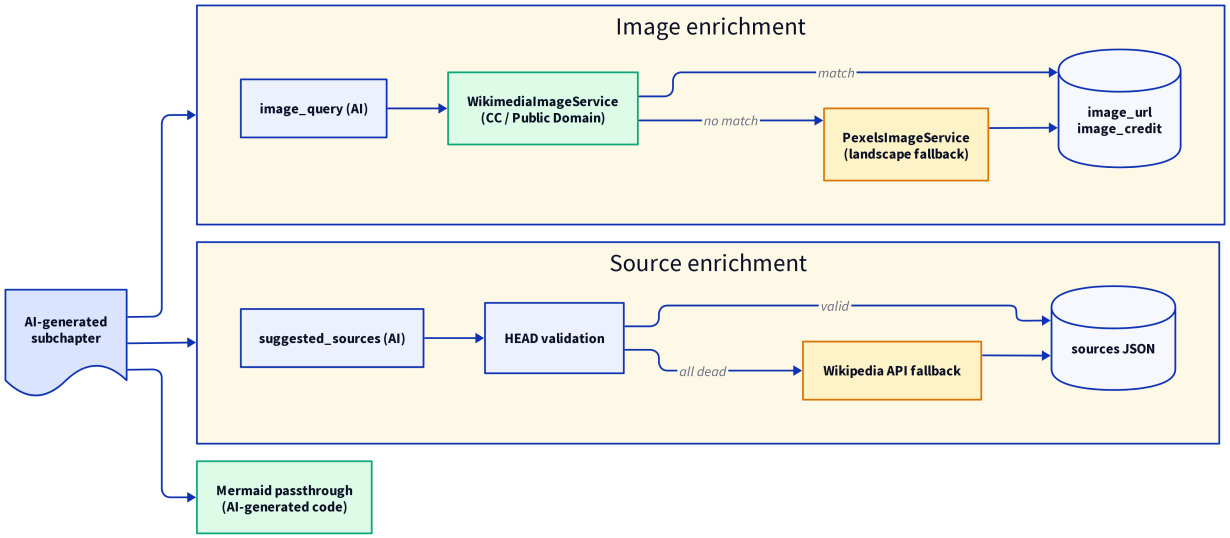

Figure 6: Stratégie duale avec fallbacks : images CC en priorité, sources validées par HEAD, repli Wikipedia

Élément	Source primaire	Fallback	Stockage
Images	Wikimedia Commons (CC)	Pexels API	`image_url, image_credit`
Sources	IA `suggested_sources` validé par HEAD	Wikipedia API par mots-clés	`sources` JSON
Diagrammes	IA Mermaid (passthrough)	—	`mermaid_code` TEXT

Services impliqués (13 au total) : `AIContentService`, `AIContentParserService`, `AIContentImportService`, `GeminiFileUploadService`, `UrlContentExtractorService`, `ImageEnrichmentService`, `WikimediaImageService`, `PexelsImageService`, `SourceFinderService`, `FlashcardService`, `QuizService`, `DashboardService`, `ContentSanitizer`.

10. Sécurité

Menace	Protection
CSRF	Tokens `@csrf` sur toutes les mutations
XSS contenu	Blade `{{ }}` + `ContentSanitizer` (whitelist HTML)
XSS Mermaid	`securityLevel: 'strict'`
IDOR	6 Policies + `Gate::authorize()` sur chaque action sensible
Layout-leak 403	`layouts.error` dédié (pas de sidebar, pas de chrome de rôle)
Mass assignment	`$fillable` whitelist sur les 14 modèles
Triche quiz	`is_correct` exclu du payload client + mélange aléatoire
Tampering quiz	Vérification serveur `answer_id` → `question_id`
Double soumission	Verrou pessimiste (`lockForUpdate`) sur l'attempt
Clés API	`.env` uniquement, jamais commités
Isolation étudiant	Chaque étudiant ne voit que ses propres générations IA et ses flashcards personnelles
Upload fichiers	MIME whitelist serveur, 10 Mo/fichier, 25 Mo total
URLs injectées	Blocklist (localhost, 127.0.0.1, réseaux privés)
Rate limiting	`throttle:10,1` sur génération AI, `throttle:20,1` sur rewrite

11. Déploiement

Élément	Configuration
Hébergeur	AlwaysData (plan gratuit)
URL	https://latreche-lms.alwaysdata.net
Document root	`www/mini-lms/public`
PHP	8.2+
BDD prod	MySQL 8
Cache	Config, routes, vues mis en cache
Assets	Tailwind, Alpine.js, Mermaid.js via CDN (pas de build)

12. Limitations Connues

Limitations identifiées

Limitation	Piste
Génération synchrone	La requête bloque le worker PHP ; Laravel Queues recommandé
Mermaid syntaxe IA	Le code généré peut contenir des erreurs ; validation basique en place
Pas de tests auto	PHPUnit/Pest non implémenté ; priorité pour v1.0
Web Speech API	Support navigateur inégal (Chrome/Edge OK, Firefox limité, Safari iOS partiel)
Images génériques	Wikimedia/Pexels parfois peu pertinents ; Mermaid compense
Backfill ownership	Les formations créées avant la migration `created_by` doivent être rattachées manuellement (migration de backfill via `activity_logs`)

13. Améliorations Futures

Axes d'évolution

	Fonctionnalité	Description
1	Laravel Queues	Génération IA + clonage de flashcards en arrière-plan
2	API REST	Endpoints pour mobile / SPA
3	Tests PHPUnit	Suite pour services critiques (`QuizService`, `FlashcardService`, `AIContentService`)
4	Notifications email	Résultats de quiz, rappels flashcards (cards `due`)
5	Export PDF	Bulletins de notes, fiches révision flashcards
6	Suivi de progression	Graphes par formation et par sous-chapitre
7	Multi-tenant	Isolation par organisation (mode SaaS)
8	Gamification	Badges, streaks de révision
9	Validation Mermaid	Pré-render serveur pour rejeter les diagrammes invalides
10	RAG	Enrichir le prompt IA avec une base de connaissances locale

Conclusion

Le projet démontre la conception et le déploiement d'une application Laravel 11 complète, articulée autour d'une architecture en couches stricte : Service Layer, Policies basées sur l'ownership, FormRequests à autorisation déléguée, et layout d'erreur isolé du dashboard. Les fonctionnalités avancées - génération IA contrôlée par Gemini, système de flashcards avec algorithme SM-2 et distribution automatique sur inscription, pipeline d'enrichissement multi-sources avec fallbacks, et parité d'édition admin/étudiant pour les contenus personnels - illustrent l'utilisation réfléchie de l'IA générative au-delà du simple appel API : le système produit du contenu structuré, validé, enrichi et réutilisable. La séparation stricte des rôles, l'absence de fuite de layout sur les erreurs et la conformité des contraintes BDD aux invariants applicatifs garantissent la robustesse en production.